

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5

Roll No.: A24126552260

Student Name: K. Sirisha

1. TITLE

Implement Modules in NodeJS

2. AIM

To implement and use custom modules in NodeJS for performing reusable mathematical operations.

3. OBJECTIVES

- 1 To understand the NodeJS module system.
- 2 To create a custom JavaScript module.
- 3 To export functions from a module.
- 4 To import a custom module using require().
- 5 To execute reusable functions from another file.

4. THEORY

NodeJS supports modular programming in which related functionality can be placed in separate files. A module can export functions or values and another file can import them. This improves code organization, reuse and maintainability.

5. ALGORITHM/PROCEDURE

- 1 Create a file named math.js for mathematical functions.
- 2 Define add(), multiply() and square() functions.
- 3 Export the functions using module.exports.
- 4 Create app.js and import the module using require('./math').
- 5 Call the exported functions with suitable values.
- 6 Display the returned values in the NodeJS terminal.
- 7 Run the program using node app.js and verify the output.

6. PROGRAM

```
// math.js
function add(a, b){
  return a + b;
}
function multiply(a, b){
  return a * b;
}
function square(a){
  return a * a;
}
module.exports = {
  add,
  multiply,
  square
};

// app.js
const math = require("./math");
const a = 8;
const b = 5;
console.log("Add:", math.add(a, b));
console.log("Multiply:", math.multiply(a, b));
console.log("Square of a:", math.square(a));
```

7. CODE EXPLANATION

Component	Purpose
function add()	Returns the sum of two numbers.
function multiply()	Returns the product of two numbers.
function square()	Returns the square of a number.
module.exports	Exports functions so another file can use them.
require('./math')	Imports the custom math module into app.js.
console.log()	Displays the calculated values in the terminal.

8. TEST CASES

The following test cases validate the exported math.js functions and the module import mechanism using require().

Test Case	Action / Input	Expected Result	Actual Result	Status
TC-01	Call math.add(8, 5) after requiring the module	Returns the sum of the two numbers (13)	Returned 13 correctly	Pass
TC-02	Call math.multiply(8, 5)	Returns the product of the two numbers (40)	Returned 40 correctly	Pass
TC-03	Call math.square(8)	Returns the square of the number (64)	Returned 64 correctly	Pass
TC-04	Import the module using require('./math')	Module loads and exposes add, multiply and square as functions	Module loaded and functions exposed correctly	Pass

8.1 TEST CASE OUTPUT SCREENSHOTS

TC-01 Output Screenshot

Call `math.add(8, 5)` after requiring the module

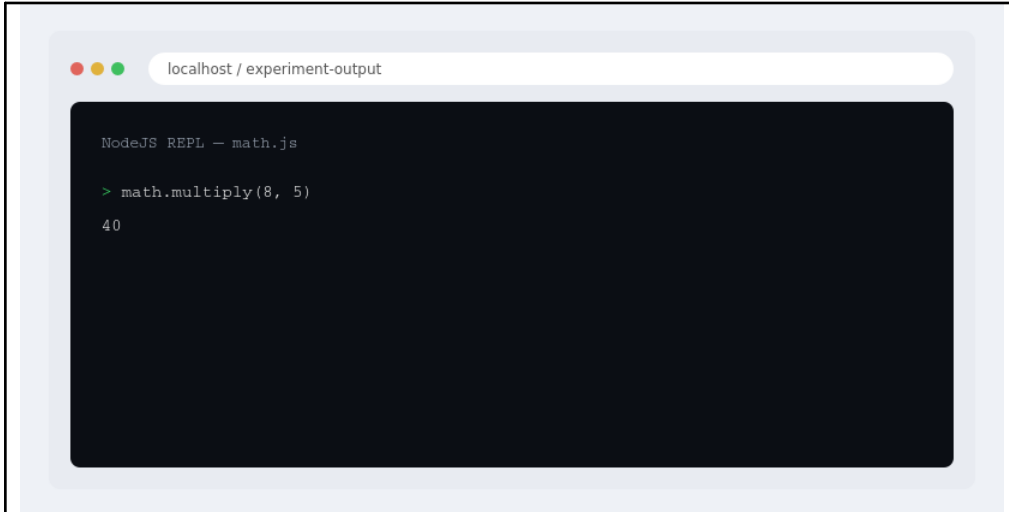


The screenshot shows a web browser window with the address bar displaying 'localhost / experiment-output'. The main content area contains a dark-themed terminal window titled 'NodeJS REPL - math.js'. The terminal shows the following sequence of commands and outputs:

```
NodeJS REPL - math.js
> const math = require('./math');
undefined
> math.add(8, 5)
13
```

TC-02 Output Screenshot

Call `math.multiply(8, 5)`



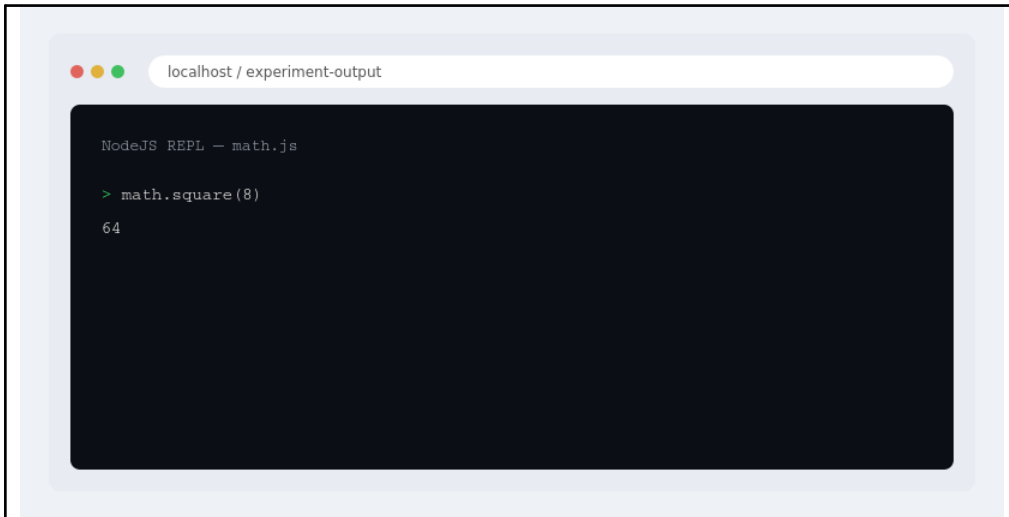
The screenshot shows a web browser window with the address bar displaying 'localhost / experiment-output'. The main content area contains a dark-themed terminal window titled 'NodeJS REPL - math.js'. The terminal shows the following sequence of commands and outputs:

```
NodeJS REPL - math.js
> math.multiply(8, 5)
40
```

8.1 TEST CASE OUTPUT SCREENSHOTS

TC-03 Output Screenshot

Call `math.square(8)`



A screenshot of a web browser window with the address bar showing 'localhost / experiment-output'. The main content area displays a NodeJS REPL session for 'math.js'. The prompt '>' is followed by the command 'math.square(8)', and the output '64' is shown below it.

```
NodeJS REPL - math.js

> math.square(8)

64
```

TC-04 Output Screenshot

Import the module using `require('./math')`



A screenshot of a web browser window with the address bar showing 'localhost / experiment-output'. The main content area displays a NodeJS REPL session for 'math.js'. The prompt '>' is followed by the command 'const math = require('./math');', and the output 'undefined' is shown. The next prompt '>' is followed by 'math', and the output is an object: '{ add: [Function: add], multiply: [Function: multiply], square: [Function: square] }'.

```
NodeJS REPL - math.js

> const math = require('./math');

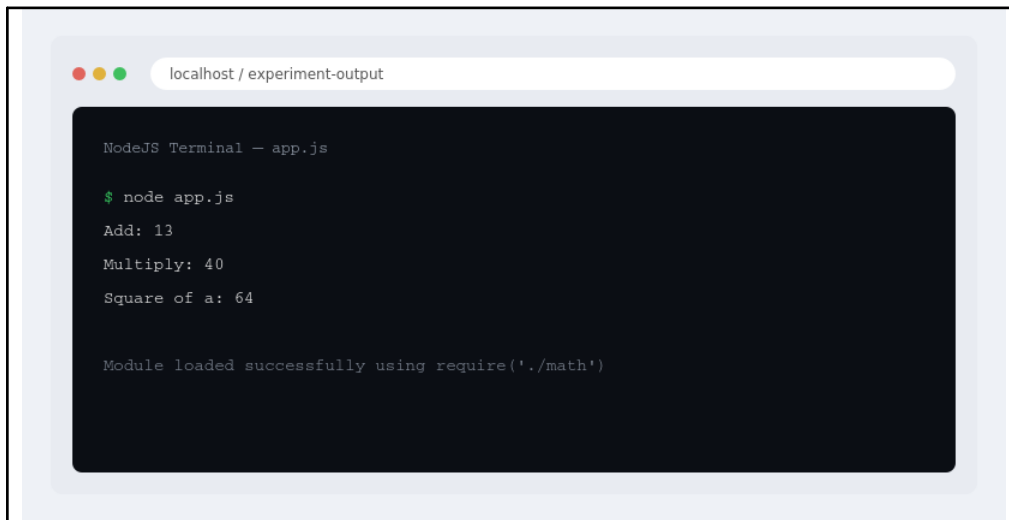
undefined

> math

{ add: [Function: add], multiply: [Function: multiply], square: [Function:
square] }
```

9. OUTPUT

Browser Output (Screenshot):

A screenshot of a web browser window. The address bar shows 'localhost / experiment-output'. The main content area displays a terminal window titled 'NodeJS Terminal - app.js'. The terminal shows the command '\$ node app.js' and its output: 'Add: 13', 'Multiply: 40', 'Square of a: 64', and 'Module loaded successfully using require('./math')'.

```
NodeJS Terminal - app.js

$ node app.js
Add: 13
Multiply: 40
Square of a: 64

Module loaded successfully using require('./math')
```

10. RESULT

Thus, a custom NodeJS module was successfully created and imported. The exported mathematical functions were reused from app.js and the expected results were displayed in the terminal.